

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I Deepak.V-192411021 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in dark ink. The signature appears to be 'Deepak' written in a cursive, slightly slanted style.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically	Generally clear with minor issues	Somewhat unclear or poorly	Disorganized and difficult to understand

		presented		structured	
--	--	-----------	--	------------	--

Question 1 — Answer

Servlet-Based Application Architecture

A servlet-based application follows a client-server architecture in which the client, usually a web browser, communicates with a web server through HTTP requests. The main components are the client, web server, servlet container, servlet, and database. The browser sends a request to the server, and the servlet container receives the request and forwards it to the appropriate servlet. The servlet processes the request and sends an HTTP response back to the browser.

The basic flow is:

Browser → Web Server → Servlet Container → Servlet → Database → Servlet → Browser

The servlet container manages servlet creation, request handling, sessions, and the servlet life cycle.

Servlet Handling GET and POST

HTML Form:

```
<form action="StudentServlet" method="post">
  <input name="name" placeholder="Name">
  <input name="email" placeholder="Email">
  <input name="course" placeholder="Course">
  <button type="submit">Register</button>
</form>
```

Java Servlet:

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req,
                          HttpServletResponse res)
        throws IOException {

        res.setContentType("text/html");
        res.getWriter().println("GET Request Received");
    }

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
        throws IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String course = req.getParameter("course");

        res.setContentType("text/html");
        res.getWriter().println("Name: " + name);
    }
}
```

```

        res.getWriter().println("Email: " + email);
        res.getWriter().println("Course: " + course);
    }
}

```

The `doGet()` method handles GET requests, while `doPost()` handles POST requests. Form parameters are obtained using `getParameter()`.

Servlet Life Cycle

- `init()`: The servlet container calls `init()` once when the servlet is created. It is used for initialization tasks.
- `service()`: The `service()` method is called for each client request. It determines the HTTP method and invokes methods such as `doGet()` or `doPost()`.
- `destroy()`: The container calls `destroy()` once before removing the servlet. It is used to release resources.

The life cycle can be represented as:

Loading → init() → service() → destroy()

GET vs POST for Sensitive Data

POST is more appropriate for sensitive information such as student credentials or passwords. GET places submitted parameters in the URL, making them potentially visible in browser history, bookmarks, and server logs. POST sends the data in the HTTP request body.

However, POST alone does not encrypt data. HTTPS must also be used to protect sensitive information during transmission.

Conclusion

Servlets provide an efficient mechanism for processing web requests and generating dynamic responses. The servlet container manages the servlet life cycle and request processing. GET is suitable for retrieving information, while POST is more appropriate for submitting sensitive or larger amounts of data, especially when combined with HTTPS.

Question 2 — Answer

Session Management Using HttpSession and Cookies

Session management is the process of maintaining information about a user across multiple HTTP requests. Since HTTP is stateless, the server does not automatically remember previous requests from the same user. Servlets provide mechanisms such as `HttpSession` and cookies to maintain user identity and session information after authentication.

Session Management Using HttpSession

After successful login, a session can be created using `getSession()`:

```

HttpSession session = request.getSession();
session.setAttribute("username", username);
response.sendRedirect("home.jsp");

```

On another page, the stored information can be retrieved using:

```
HttpSession session = request.getSession();
String user = (String) session.getAttribute("username");
```

The session can be terminated during logout:

```
session.invalidate();
```

Session Management Using Cookies

A cookie is a small piece of data stored by the browser and sent with subsequent requests to the same server.

```
Cookie cookie = new Cookie("username", username);
response.addCookie(cookie);
```

The cookie can later be retrieved from the request:

```
Cookie[] cookies = request.getCookies();

if(cookies != null){
    for(Cookie c : cookies){
        if(c.getName().equals("username"))
            System.out.println(c.getValue());
    }
}
```

For authentication, it is generally safer to store a session identifier in a cookie rather than directly storing sensitive information.

How Session Tracking Works

The basic process is:

Login → Create Session → Generate Session ID → Send ID to Browser → Browser Returns ID → Server Identifies Session → Access Protected Pages

A common mechanism is the JSESSIONID session identifier. It allows the server to associate subsequent requests with the correct session.

If cookies are unavailable, URL rewriting can also be used as an alternative session-tracking mechanism.

Comparison Between HttpSession and Cookies

HttpSession:

- Session information is mainly stored on the server.
- It is generally safer for sensitive session data.
- The session can be invalidated by the server.
- It is commonly used for login and authentication information.

Cookies:

- Data is stored on the client browser.
- Cookies have limited storage capacity.
- They can be session or persistent cookies.
- They are commonly used for preferences and session identifiers.

Security Considerations

Session management should use HTTPS to protect session identifiers during transmission. Authentication cookies should use appropriate security attributes such as Secure and HttpOnly. Sessions should also expire after inactivity, and session identifiers should be regenerated after successful authentication to reduce session fixation risks.

Conclusion

HttpSession provides convenient server-side session management, while cookies provide a mechanism for maintaining information between requests. In servlet-based authentication systems, a common approach is to maintain user information on the server using HttpSession and use a secure cookie containing the session identifier to associate the browser with that session.

Question 3 — Answer

JDBC-Based Product Management Application

JDBC (Java Database Connectivity) is a standard Java API used to connect Java applications with relational databases. A JDBC-based web application can allow an e-commerce platform to insert, retrieve, update, and delete product information using SQL queries.

JDBC Architecture

The main components of JDBC architecture are:

Java Application → JDBC API → JDBC Driver → Database

The Java application uses JDBC classes and interfaces such as Connection, PreparedStatement, and ResultSet. The JDBC driver converts Java database requests into a format understood by the specific database system.

Database Table

```
CREATE TABLE product(  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

JDBC Connection and Product Operations

```
import java.sql.*;  
  
public class ProductDB {  
  
    static final String URL =  
        "jdbc:mysql://localhost:3306/shop";  
    static final String USER = "root";  
    static final String PASS = "password";  
  
    public static void main(String[] args) {  
  
        String insert =  
            "INSERT INTO product VALUES (?, ?, ?)";  
  
        String select =
```

```

        "SELECT * FROM product";

        try(Connection con =
            DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps =
                con.prepareStatement(insert)) {

            ps.setInt(1, 1);
            ps.setString(2, "Laptop");
            ps.setDouble(3, 55000);

            ps.executeUpdate();

            try(PreparedStatement stmt =
                con.prepareStatement(select);
                ResultSet rs = stmt.executeQuery()) {

                while(rs.next()) {
                    System.out.println(
                        rs.getInt("id") + " " +
                        rs.getString("name") + " " +
                        rs.getDouble("price")
                    );
                }
            }

        } catch(SQLException e) {
            System.out.println(
                "Database Error: " + e.getMessage()
            );
        }
    }
}

```

Steps Involved in JDBC Connectivity

- **Load/Register Driver:** The JDBC driver makes communication between Java and the database possible.
- **Establish Connection:** `DriverManager.getConnection()` establishes a connection with the database.
- **Create Statement:** `PreparedStatement` is used to prepare SQL queries safely.
- **Execute Query:** `executeUpdate()` is used for INSERT, UPDATE, and DELETE operations, while `executeQuery()` is used for SELECT operations.
- **Process Result:** The `ResultSet` object contains the records returned by a SELECT query.
- **Close Resources:** Connections, statements, and result sets must be closed after use. Try-with-resources can automatically close them.

Exception Handling

JDBC operations can produce `SQLException` because of connection failures, invalid SQL statements, unavailable databases, or constraint violations. Therefore, database operations should be placed inside a try-catch block. Proper resource management should also be used to prevent connection leaks.

`PreparedStatement` is preferred over dynamically constructing SQL strings because it helps prevent SQL injection and handles parameters safely.

Conclusion

JDBC provides a standard method for connecting Java applications to relational databases. The basic process involves establishing a connection, preparing SQL statements, executing queries, processing results, handling exceptions, and closing resources. Using PreparedStatement, proper exception handling, and resource management makes an e-commerce product management application more secure, reliable, and maintainable.